Zippy Mock Logistics Platform – Coding Assignment

Objective

Build a simplified logistics aggregation platform that allows a merchant to:

1. Create an order.
2. Request shipping options from three mock courier services.
3. Compare courier rates and estimated delivery times.
4. Select a courier.
5. Create a shipment with the selected courier.
6. Receive shipment-status updates from the courier.
7. Display the latest order and shipment status in the frontend.

The application should simulate how a logistics aggregator such as Zippy connects merchants with multiple courier companies.

Functional Flow

Step 1: Create an Order

The frontend should provide a form to create an order.

The form should collect:

- Merchant order number
- Customer name
- Customer phone number
- Customer email
- Pickup address
- Delivery address
- Pickup pincode
- Delivery pincode
- Package weight
- Package dimensions
- Payment type

- COD amount, when applicable

Sample Order Request

```
{
 "merchantOrderId": "MERCHANT-10001",
 "customer": {
  "name": "Rahul Sharma",
  "phone": "9876543210",
  "email": "rahul@example.com"
 },
 "pickupAddress": {
  "addressLine1": "15 MG Road",
  "city": "Bengaluru",
  "state": "Karnataka",
  "pincode": "560001"
 },
 "deliveryAddress": {
  "addressLine1": "22 Connaught Place",
  "city": "New Delhi",
  "state": "Delhi",
  "pincode": "110001"
 },
 "package": {
  "weightGrams": 1500,
  "lengthCm": 20,
  "widthCm": 15,
  "heightCm": 10
 },
 "paymentType": "COD",
 "codAmount": 2500
}
```

The Zippy backend should save the order with an initial status:

ORDER_CREATED

The backend should generate a Zippy order ID such as:

ZPY-ORD-10001

Step 2: Get Rates from Three Mock Courier Services

After the order is created, the backend should request rates from three mock courier services.

Each courier service must expose its own API and return data in a different JSON format.

The candidate may build the mock courier services as:

- Three separate Spring Boot services,
- Three lightweight Node.js services,
- Three endpoints inside one mock-carrier application,
- Or another clearly separated implementation.

The Zippy backend must not expose the courier-specific formats directly to the frontend.

It should convert all courier responses into a common Zippy response format.

Courier 1: FastShip

Rate API

POST /fastship/api/v1/rate

Request

```
{
 "origin_pin": "560001",
 "destination_pin": "110001",
 "weight_kg": 1.5,
 "payment_mode": "COD",
 "invoice_value": 2500
}
```

Response

```
{
 "success": true,
 "service": {
  "service_code": "FAST-AIR",
  "service_name": "FastShip Air Express",
  "freight_charge": 120.00,
  "cod_charge": 35.00,
  "tax": 27.90,
  "total_amount": 182.90,
```

```
   "estimated_days": 2
  }
}
```

Courier 2: QuickExpress

Rate API

POST /quickexpress/rates/check

Request

```
{
  "pickupPincode": "560001",
  "deliveryPincode": "110001",
  "weightInGrams": 1500,
  "dimensions": {
    "length": 20,
    "breadth": 15,
    "height": 10
  },
  "isCod": true,
  "collectableAmount": 2500
}
```

Response

```
{
  "status": "AVAILABLE",
  "quoteId": "QE-Q-90001",
  "charges": {
    "shipping": 115.00,
    "cod": 40.00,
    "fuelSurcharge": 12.00,
    "gst": 30.06
  },
  "payable": 197.06,
  "deliveryEstimate": {
    "minimumDays": 2,
    "maximumDays": 3
  },
  "product": "EXPRESS"
}
```

Courier 3: ReliableCourier

Rate API

GET /reliablecourier/shipping-options

Query Parameters

from=560001
to=110001
weight=1500
cod=true
amount=2500

Response


```
{
 "code": 200,
 "data": [
  {
   "id": "RC-SURFACE",
   "name": "Reliable Surface",
   "rate": {
    "base": 95.00,
    "handling": 10.00,
    "cashCollectionFee": 30.00,
    "taxAmount": 24.30,
    "grandTotal": 159.30
   },
   "eta": "4-5 business days"
  },
  {
   "id": "RC-AIR",
   "name": "Reliable Air",
   "rate": {
    "base": 130.00,
    "handling": 12.00,
    "cashCollectionFee": 30.00,
    "taxAmount": 30.96,
    "grandTotal": 202.96
   },
   "eta": "2-3 business days"
```

```
  }
 ]
}
```

Step 3: Normalize the Courier Responses

The Zippy backend should convert all courier responses into one common format.

Zippy Rate Response

```
{
 "orderId": "ZPY-ORD-10001",
 "shippingOptions": [
  {
   "carrierCode": "FASTSHIP",
   "carrierName": "FastShip",
   "serviceCode": "FAST-AIR",
   "serviceName": "FastShip Air Express",
   "baseCharge": 120.00,
   "codCharge": 35.00,
   "additionalCharges": 0.00,
   "tax": 27.90,
   "totalCharge": 182.90,
   "estimatedMinDays": 2,
   "estimatedMaxDays": 2
  },
  {
   "carrierCode": "QUICKEXPRESS",
   "carrierName": "QuickExpress",
   "serviceCode": "EXPRESS",
   "serviceName": "QuickExpress Express",
   "baseCharge": 115.00,
   "codCharge": 40.00,
   "additionalCharges": 12.00,
   "tax": 30.06,
   "totalCharge": 197.06,
   "estimatedMinDays": 2,
   "estimatedMaxDays": 3
  },
  {
   "carrierCode": "RELIABLE",
```

```
    "carrierName": "ReliableCourier",
    "serviceCode": "RC-SURFACE",
    "serviceName": "Reliable Surface",
    "baseCharge": 95.00,
    "codCharge": 30.00,
    "additionalCharges": 10.00,
    "tax": 24.30,
    "totalCharge": 159.30,
    "estimatedMinDays": 4,
    "estimatedMaxDays": 5
   }
 ]
}
```


The frontend should display the options in a table.

Suggested columns:

Carrier Service	Base Charge	COD Charge	Other Charges	Tax Total	Estimated Delivery	Select

The frontend should allow the user to sort or compare rates by:

- Lowest price
- Fastest delivery
- Carrier name

Step 4: Select a Courier

The merchant should select one shipping option.

Zippy API

POST /api/orders/{orderId}/select-carrier

Request


```
{
  "carrierCode": "FASTSHIP",
  "serviceCode": "FAST-AIR",
  "quotedAmount": 182.90
}
```


The backend should save:

- Selected carrier
- Selected service
- Quoted shipping charge
- Selection timestamp
- Original carrier quote
- Current shipment status

The order status should change to:

CARRIER_SELECTED

Step 5: Notify the Selected Courier and Create Shipment

After the carrier is selected, the Zippy backend should call the selected courier's shipment-creation API.

Each courier should again use a different request and response format.

FastShip Shipment Creation

API

POST /fastship/api/v1/shipments

Request

```
{
 "reference_number": "ZPY-ORD-10001",
 "service_code": "FAST-AIR",
 "shipper": {
  "name": "Zippy Merchant",
  "postal_code": "560001"
 },
 "consignee": {
  "name": "Rahul Sharma",
  "phone": "9876543210",
  "postal_code": "110001"
 },
 "parcel": {
  "weight_kg": 1.5
 },
```

```
 "cod": {
  "enabled": true,
  "amount": 2500
 },
 "callback_url": "http://zippy-backend/api/webhooks/fastship"
}
```


Response


```
{
 "success": true,
 "shipment_id": "FS-700001",
 "tracking_number": "FST123456789",
 "label_url": "http://mock-fastship/labels/FST123456789.pdf",
 "status": "BOOKED"
}
```

QuickExpress Shipment Creation

API

POST /quickexpress/booking/create

Request


```
{
 "clientOrderId": "ZPY-ORD-10001",
 "quoteId": "QE-Q-90001",
 "productType": "EXPRESS",
 "receiverDetails": {
  "fullName": "Rahul Sharma",
  "mobileNumber": "9876543210",
  "postalCode": "110001"
 },
 "packageDetails": {
  "deadWeight": 1500,
  "weightUnit": "GRAM"
 },
 "payment": {
  "mode": "CASH_ON_DELIVERY",
  "amountToCollect": 2500
 },
```

```
  "webhook": "http://zippy-backend/api/webhooks/quickexpress"
}
```

Response

```
{
  "bookingStatus": "CONFIRMED",
  "booking": {
    "bookingId": "QE-B-800001",
    "awb": "QE987654321",
    "currentState": "SHIPMENT_CREATED"
  }
}
```

ReliableCourier Shipment Creation

API

PUT /reliablecourier/orders

Request

```
{
  "orderReference": "ZPY-ORD-10001",
  "selectedOption": "RC-SURFACE",
  "destination": {
    "contact": "Rahul Sharma",
    "phone": "9876543210",
    "zip": "110001"
  },
  "parcelWeight": {
    "value": 1.5,
    "unit": "KG"
  },
  "collectionType": "COD",
  "collectionAmount": 2500,
  "statusNotificationUrl": "http://zippy-backend/api/webhooks/reliable"
}
```

Response

```
{
  "result": "ACCEPTED",
  "deliveryOrder": {
```

```
   "id": "RC-DO-600001",
   "trackingCode": "RC1122334455"
  },
  "message": "Shipment successfully registered"
}
```


After successful shipment creation, Zippy should store the carrier shipment ID and tracking number.

The order status should change to:

SHIPMENT_CREATED

Step 6: Courier Status Events

The mock courier should send shipment updates to the Zippy backend.

The courier may automatically send events on a timer, or the candidate may provide a mock courier screen or API to trigger the next status.

Suggested simulated flow:

SHIPMENT_CREATED
PICKED_UP
IN_TRANSIT
OUT_FOR_DELIVERY
DELIVERED

The mock carrier should call a Zippy webhook endpoint whenever the shipment status changes.

FastShip Webhook Format

Zippy Endpoint

POST /api/webhooks/fastship

Payload


```
{
  "shipment_id": "FS-700001",
  "tracking_number": "FST123456789",
  "event_code": "IN_TRANSIT",
  "event_description": "Shipment departed Bengaluru hub",
  "event_time": "2026-07-15T10:30:00Z",
```

```
  "location": "Bengaluru Hub"
}
```

QuickExpress Webhook Format

Zippy Endpoint

POST /api/webhooks/quickexpress

Payload

```
{
  "awb": "QE987654321",
  "event": {
    "type": "OFD",
    "message": "Shipment is out for delivery",
    "occurredAt": "2026-07-17T08:15:00Z"
  },
  "facility": {
    "city": "New Delhi",
    "code": "DEL-01"
  }
}
```

QuickExpress event codes may be:

SC = Shipment Created
PU = Picked Up
IT = In Transit
OFD = Out for Delivery
DLV = Delivered

ReliableCourier Webhook Format

Zippy Endpoint

POST /api/webhooks/reliable

Payload

```
{
  "trackingCode": "RC1122334455",
  "statusId": 50,
  "statusText": "Delivered",
```

```
 "updatedOn": "2026-07-19T15:45:00Z",
 "proofOfDelivery": {
  "receivedBy": "Rahul Sharma",
  "deliveryLocation": "New Delhi"
 }
}
```


ReliableCourier status IDs may be:

10 = Shipment Created
20 = Picked Up
30 = In Transit
40 = Out for Delivery
50 = Delivered
60 = Delivery Failed
70 = Returned to Origin

Step 7: Normalize Shipment Statuses

The Zippy backend should map all courier-specific statuses to common Zippy statuses.

Zippy Status	FastShip	QuickExpress	ReliableCourier
SHIPMENT_CREATED	BOOKED	SC	10
PICKED_UP	PICKED_UP	PU	20
IN_TRANSIT	IN_TRANSIT	IT	30
OUT_FOR_DELIVERY	OUT_FOR_DELIVERY	OFD	40
DELIVERED	DELIVERED	DLV	50
DELIVERY_FAILED	DELIVERY_FAILED	NDR	60
RTO	RETURNED	RTO	70

The backend should maintain both:

- Current shipment status
- Complete shipment-status history

Example Status History


```
[
 {
```

```
  "status": "SHIPMENT_CREATED",
  "description": "Shipment created with FastShip",
  "eventTime": "2026-07-15T09:00:00Z"
 },
 {
  "status": "PICKED_UP",
  "description": "Shipment picked up from merchant",
  "eventTime": "2026-07-15T14:00:00Z"
 },
 {
  "status": "IN_TRANSIT",
  "description": "Shipment departed Bengaluru hub",
  "eventTime": "2026-07-16T10:30:00Z"
 }
]
```

Step 8: Update the Frontend

The frontend should contain at least three screens.

1. Create Order Screen

Allows the merchant to enter order and package information.

2. Courier Selection Screen

Displays normalized rates from all available carriers and allows the merchant to select one.

3. Order Details and Tracking Screen

Displays:

- Zippy order ID
- Merchant order ID
- Customer details
- Selected carrier
- Selected service
- Shipping charge
- Tracking number
- Current shipment status

- Shipment-status history

The frontend may update shipment status using one of the following:

- Polling every few seconds
- Server-Sent Events
- WebSocket
- Manual refresh

WebSocket or Server-Sent Events may be treated as a bonus feature.

Suggested Zippy APIs

POST /api/orders
GET /api/orders/{orderId}
POST /api/orders/{orderId}/rates
GET /api/orders/{orderId}/rates
POST /api/orders/{orderId}/select-carrier
POST /api/orders/{orderId}/create-shipment
GET /api/orders/{orderId}/tracking
POST /api/webhooks/fastship
POST /api/webhooks/quickexpress
POST /api/webhooks/reliable

The candidate may combine carrier selection and shipment creation into one API, but should explain the design decision.

Minimum Data Model

Order

id
zippy_order_id
merchant_order_id
customer_name
customer_phone
customer_email
pickup_pincode
delivery_pincode
weight_grams
length_cm

width_cm
height_cm
payment_type
cod_amount
order_status
created_at
updated_at

Shipping Quote

id
order_id
carrier_code
service_code
service_name
base_charge
cod_charge
additional_charges
tax
total_charge
estimated_min_days
estimated_max_days
raw_carrier_response
created_at

Shipment

id
order_id
carrier_code
carrier_shipment_id
tracking_number
selected_service_code
quoted_amount
current_status
created_at
updated_at

Shipment Event

id
shipment_id
carrier_event_id
carrier_status

normalized_status
description
location
event_time
raw_event_payload
received_at

Important Technical Requirements

The solution should:

1. Use a frontend framework such as React, Angular, or Vue.
2. Use a backend framework such as Spring Boot, Node.js, .NET, or another approved framework.
3. Save orders, quotes, shipments, and events in a database.
4. Integrate with all three mock courier services.
5. Normalize different courier request and response formats.
6. Handle one courier being unavailable without failing the entire rate request.
7. Prevent duplicate webhook events from creating duplicate status-history records.
8. Validate order and shipment requests.
9. Return appropriate HTTP status codes.
10. Include clear setup instructions.

Failure Scenarios to Handle

Candidates should handle at least the following:

One Carrier Is Down

If QuickExpress returns an HTTP 500 error, rates from FastShip and ReliableCourier should still be returned.

Carrier Timeout

The Zippy backend should not wait indefinitely for a courier response.

The candidate should configure a reasonable timeout.

Duplicate Webhook

If a courier sends the same event more than once, Zippy should not create duplicate history entries.

Unknown Tracking Number

A webhook received for an unknown shipment should be rejected or safely recorded for investigation.

Invalid Status Transition

A shipment should not normally move from:

DELIVERED

back to:

IN_TRANSIT

The candidate should either reject the update or explain how such exceptions would be handled.

Price Change

The selected carrier quote should be saved with the shipment so later rate changes do not alter the original quoted amount.

Testing Expectations

At minimum, include tests for:

- Order creation
- Courier-response normalization
- Rate sorting
- One courier failing
- Carrier selection
- Shipment creation
- Webhook status mapping
- Duplicate webhook handling
- Invalid status transition

Deliverables

The candidate should provide:

1. Source code.
2. Database schema or migration scripts.
3. README with setup instructions.
4. Sample test data.
5. API documentation or Postman collection.
6. Unit tests.
7. A short explanation of the application architecture.
8. Instructions for triggering courier-status events.

Docker Compose is preferred so that the frontend, Zippy backend, database, and mock